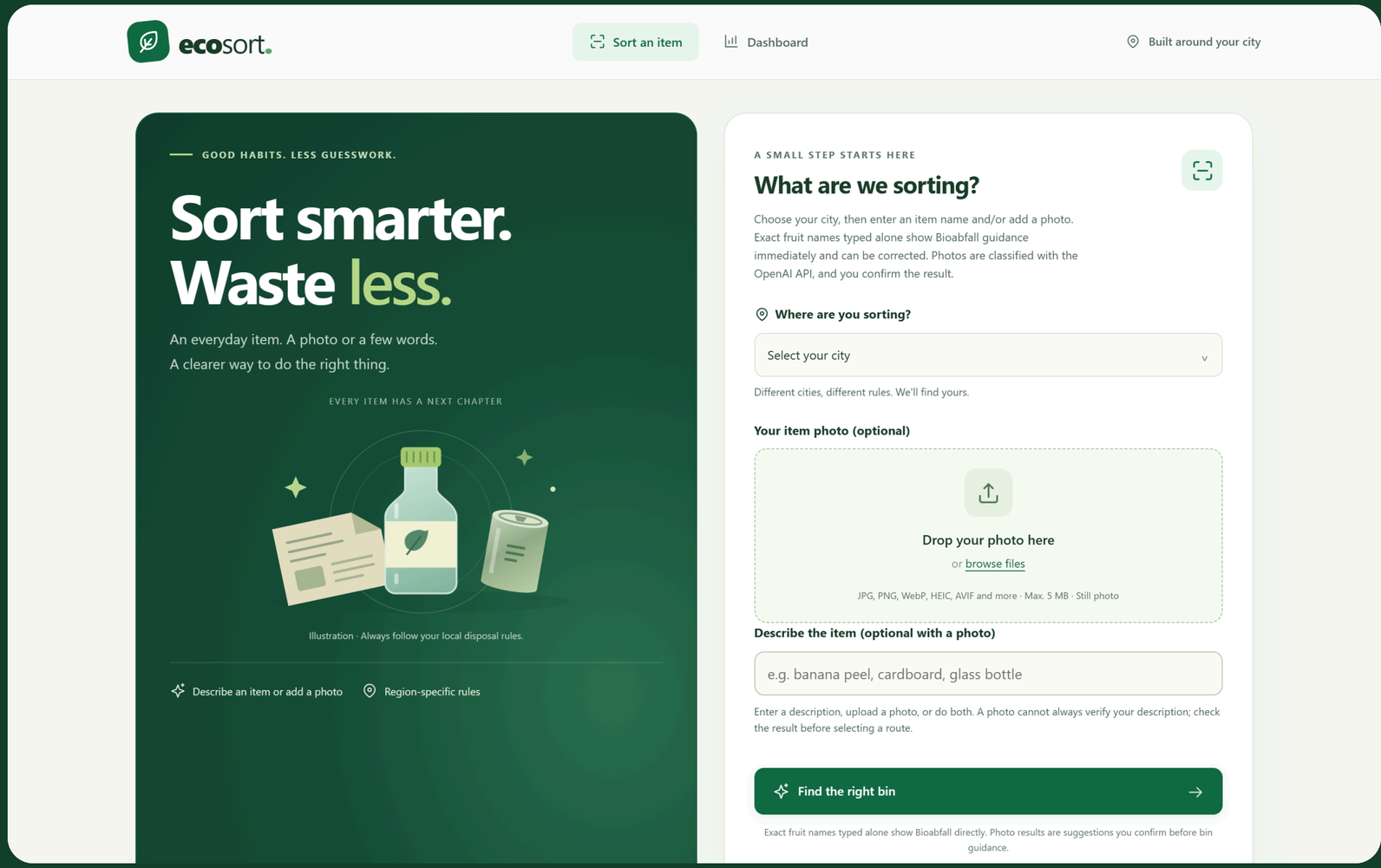
ecosort.

AI-Powered Waste Sorting and Recycling Assistant

Capstone project, final presentation

Oluwatimilehin Ogunme · Terkuma Caleb Lumun · Onyedikachi Onah · Moye Nyuysoni Glein Perry

Compact Programming Course with Python · Fachhochschule Dortmund · 21 November 2026



Team and subsystems

A

AI Services

Oluwatimilehin Ogunme

Product owner, scrum master

Classifies photos with the OpenAI API and checks every answer.

`ai_services.py`
`ai_prompts.py`
`pipeline.py`

B

Data & Rules

Terkuma Caleb Lumun

Database and city rules

Stores scans and loads each city's disposal rules.

`db.py`
`schema.sql`,
`schema_mysql.sql`
`data/rules_seed.csv`

C

Dashboard & Analytics

Onyedikachi Onah

Statistics and charts

Turns saved scans into statistics and charts.

`analytics.py`

D

Web App & Integration

Moye Nyuysoni Glein Perry

Integration and deployment

Flask web app that connects A, B and C, plus the review flow and offline photo matching.

`app/__init__.py, routes/`
`templates/, Dockerfile`
`scan_reviews.py`
`disposal.py, item_text.py`
`image_uploads.py`
`local_vision.py`

Domain

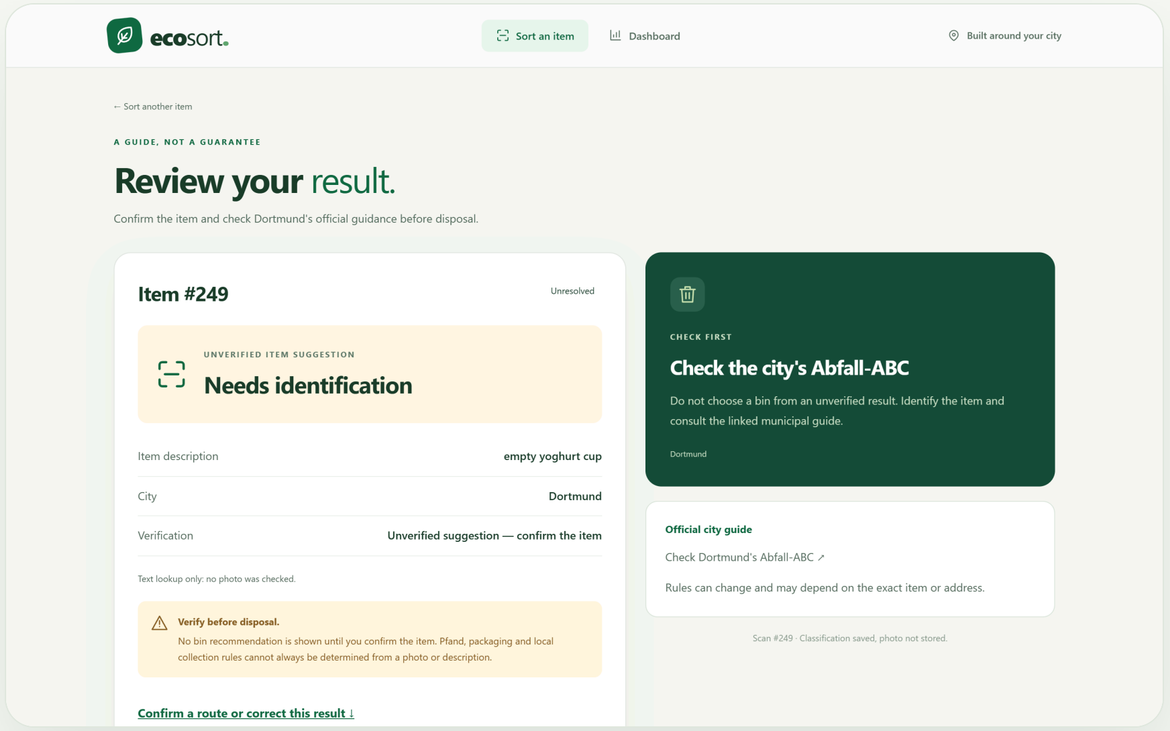
Household waste sorting for residents and municipalities in Dortmund, Berlin and Munich.

1. **A resident describes an item or uploads a photo**
   OpenAI vision or a text lookup suggests what it is.

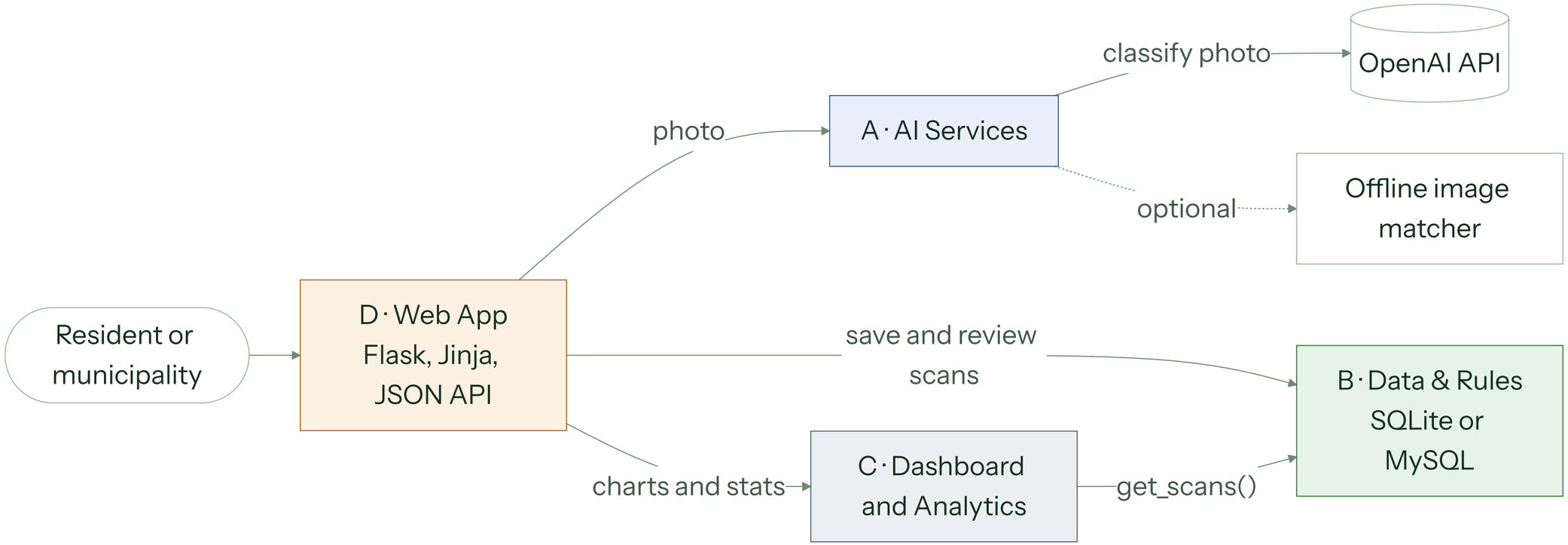
2. **The resident confirms or corrects the route**
   No bin is shown for an AI suggestion until it is confirmed.

3. **The app shows the city's bin and instructions**
   Plus a link to the city's official Abfall-ABC.

4. **The municipality sees what is being sorted**
   Dashboard per region and date range.



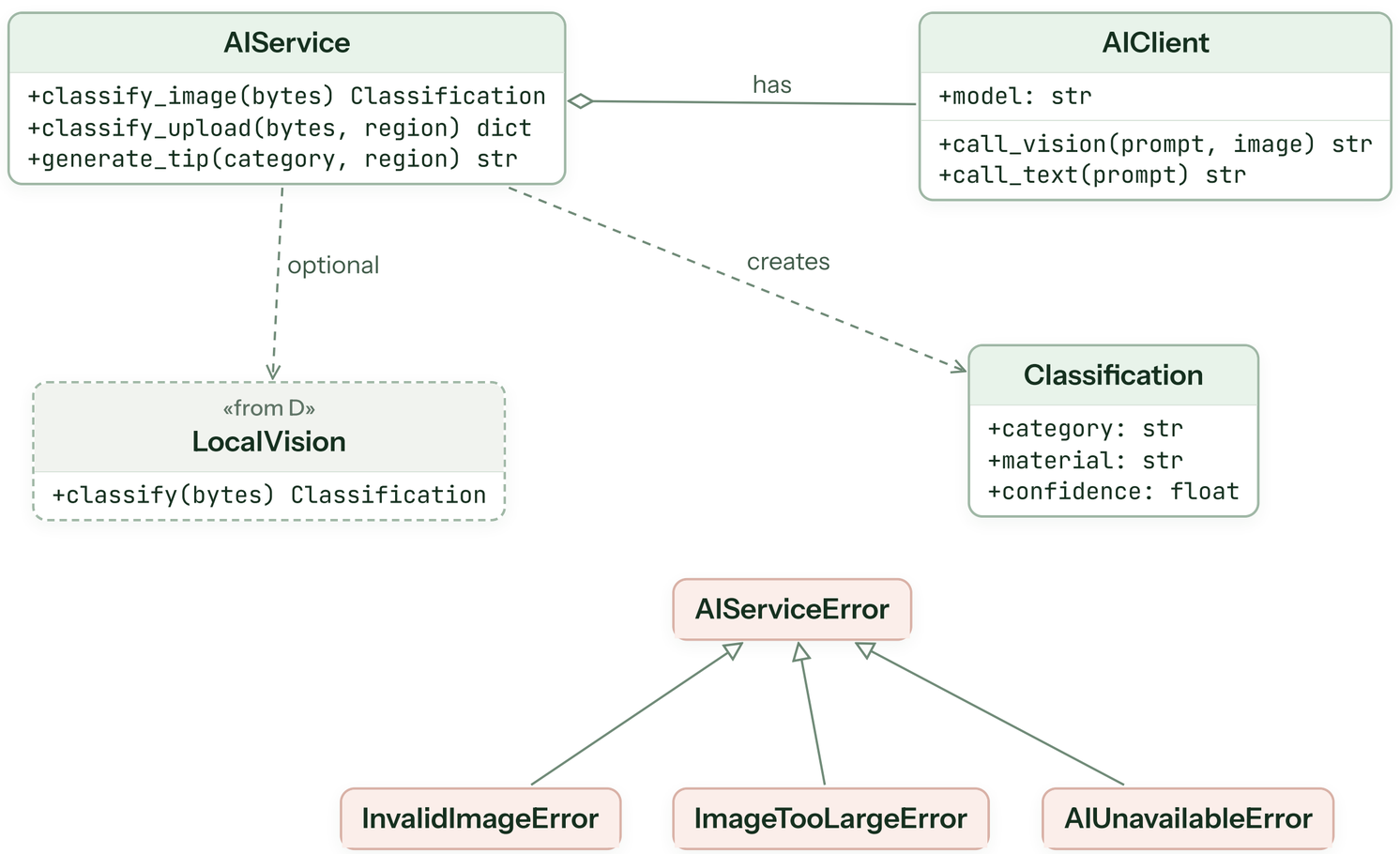
System architecture



Requirements and traceability

Owner	Functional
A D	Classify an item from a photo or a name into recyclable, compostable, landfill or unknown
A D	Ask the user to confirm or correct every AI suggestion
A D	Retry a failed AI call once, then let the user choose the bin by hand
B	Save every scan: region, category, material, confidence, time
B	Load disposal rules per city from a CSV file
C	Show scans per category and the recycling rate, by city and date
D	Show the city's bin, instructions and official guide link
D	Web pages for sorting, result and dashboard, plus a JSON API

Owner	Non-functional
A D	API key only on the server, never in Git
B	Parameterised SQL only, SQLite or MySQL
C	Same colour per category in every chart
D	Runs with Docker Compose (app + MySQL)
D	No stack traces shown to users
A B C D	Unit tests run without network or a real key



How it works

1. **Check the photo**
   JPEG or PNG, max. 5 MB. D converts other formats first.
2. **Ask the vision model for JSON**
   category, material, confidence
3. **Validate the answer**
   No item name means "unknown", never a guessed bin.
4. **Return a dict to D**
   generate_tip() can add a short tip, max. 300 characters.

```python
# ai_services.py · AIService.classify_image
answer = self._with_one_retry(
    lambda: self._client.call_vision(CLASSIFY_PROMPT, image_bytes, mime_type))
data = json.loads(answer)
category = data.get("category", "unknown")
if category not in CATEGORIES:
    category = "unknown"
material = data.get("material")
if not isinstance(material, str) or not material.strip():
    return Classification("unknown", "unidentified item", 0.0)
return Classification(category, material, float(data.get("confidence", 0.0)))

# one retry, then a clear error for the user
def _with_one_retry(call):
    try:
        return call()
    except Exception:
        logger.info("Retrying the OpenAI call once")
    try:
        return call()
    except Exception as error:
        raise AIUnavailableError("AI is not available, ...") from error
```

Output validation

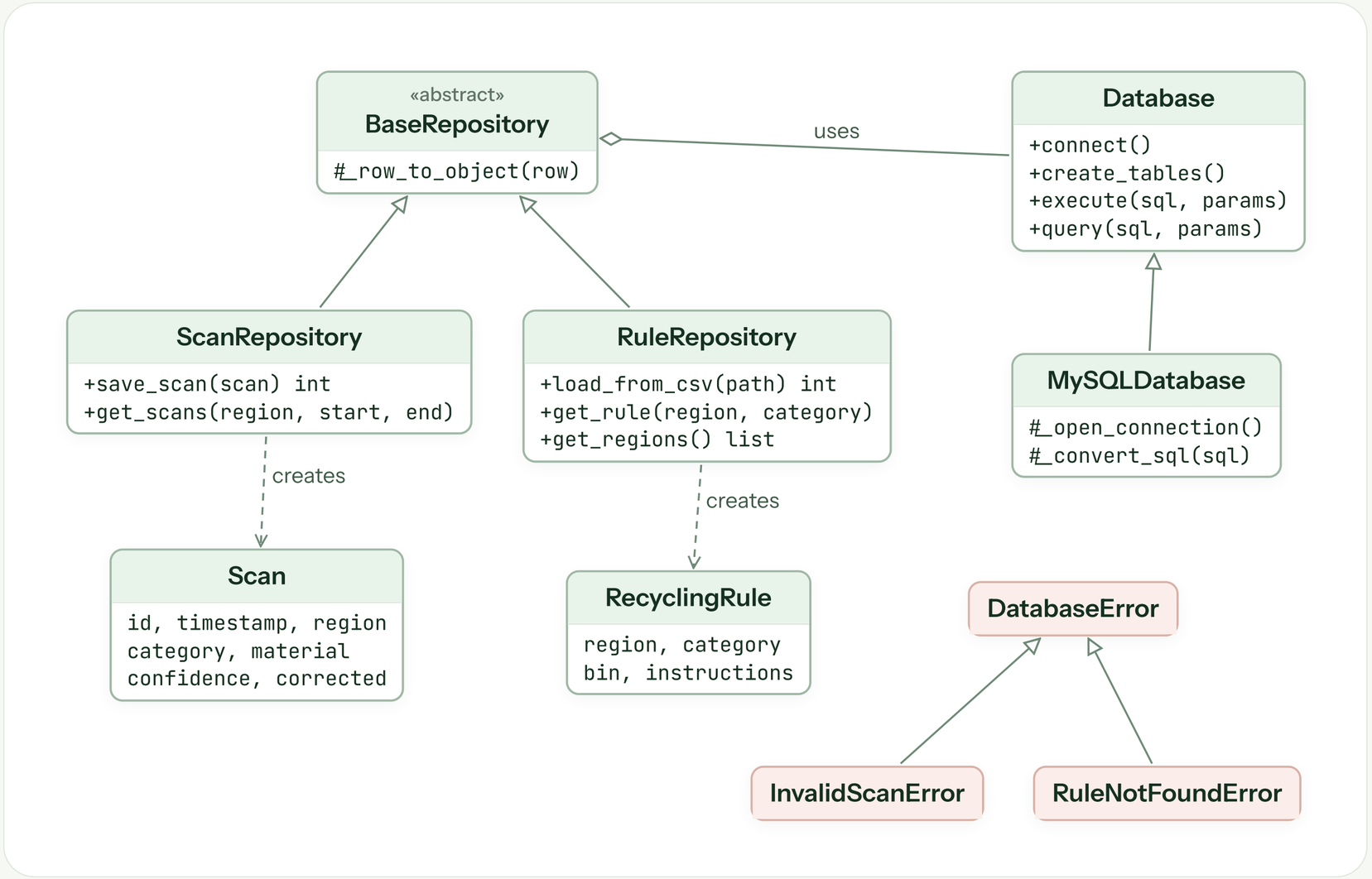
Unknown categories and missing names fall back to "unknown".

Error handling

One retry, then a message and manual choice. No stack trace.

Testing

The AI client is mocked, so tests need no key.



How it works

1. **Create the tables on first start**
   One schema file per database.
2. **Load city rules from CSV**
   Dortmund, Berlin, Munich. Editable in a spreadsheet.
3. **Validate and save each scan**
   Setters reject invalid values.
4. **Filter scans by city and date**
   C reads scans only through get_scans().

```python
# db.py · ScanRepository
class ScanRepository(BaseRepository):
    def save_scan(self, scan: Scan) → int:
        if scan.id is not None:
            raise InvalidScanError(f"Scan already saved with id {scan.id}")
        new_id = self._execute(
            "INSERT INTO scans (timestamp, region, category,"
            " material, confidence, corrected)"
            " VALUES (?, ?, ?, ?, ?, ?)",
            (
                scan.timestamp.isoformat(),
                scan.region,
                scan.category,
                scan.material,
                scan.confidence,
                int(scan.corrected),
            ),
        )
        scan.id = new_id
        return new_id
```

Parameterised SQL

Placeholders only. User input never becomes SQL text.

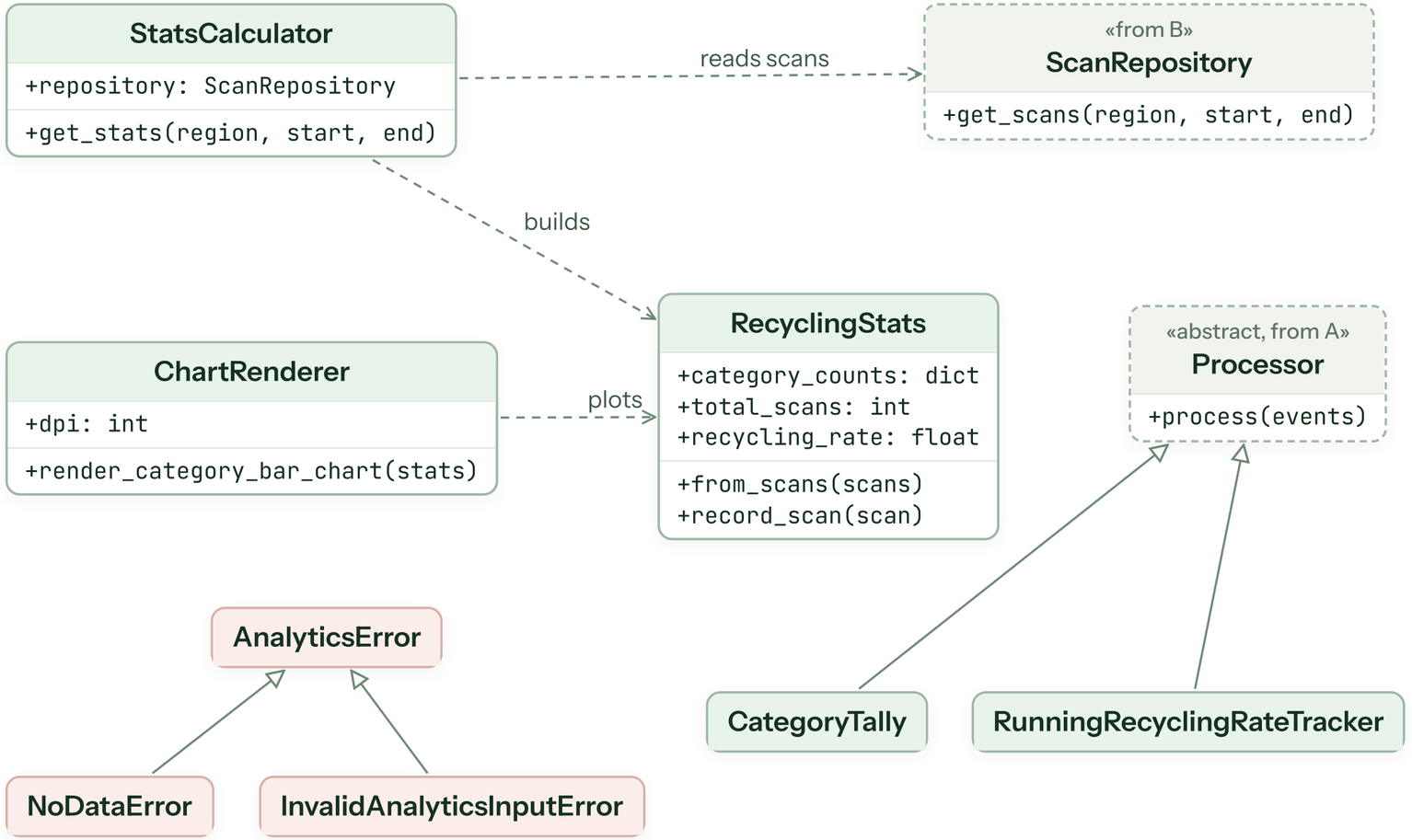
Read access

C reads scans only through get_scans().

SQLite and MySQL

SQLite locally, MySQL in Docker. Same repositories, one schema file each.

C Dashboard & Analytics · Onyedikachi Onah



How it works

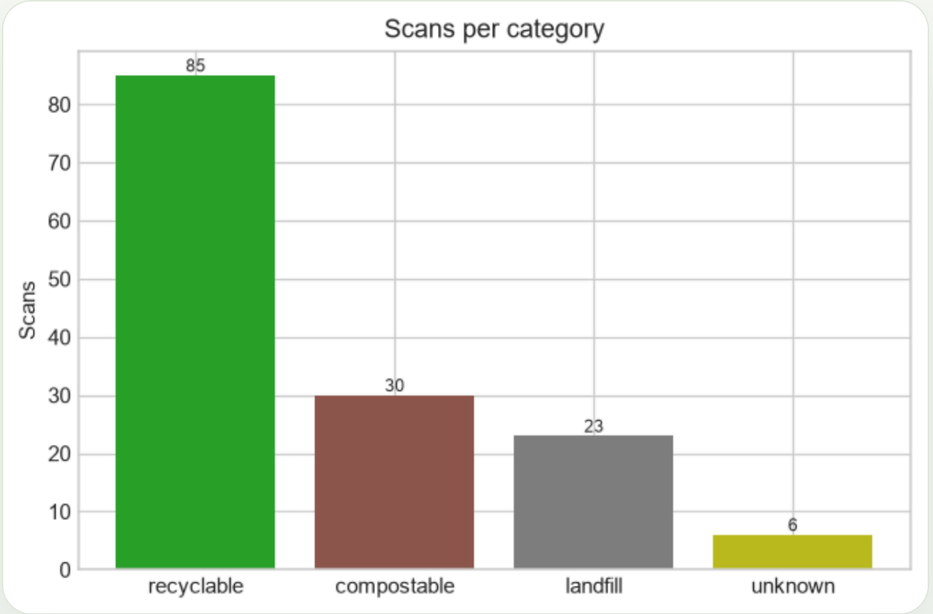
1. **Read scans through B**
   get_scans(region, start, end), never SQL.
2. **Count each category**
   Unknown categories raise an error.
3. **Calculate the recycling rate**
   Recyclable + compostable share of scans. The dashboard leaves out unknown items.
4. **Render the chart as PNG**
   matplotlib, same colour per category.

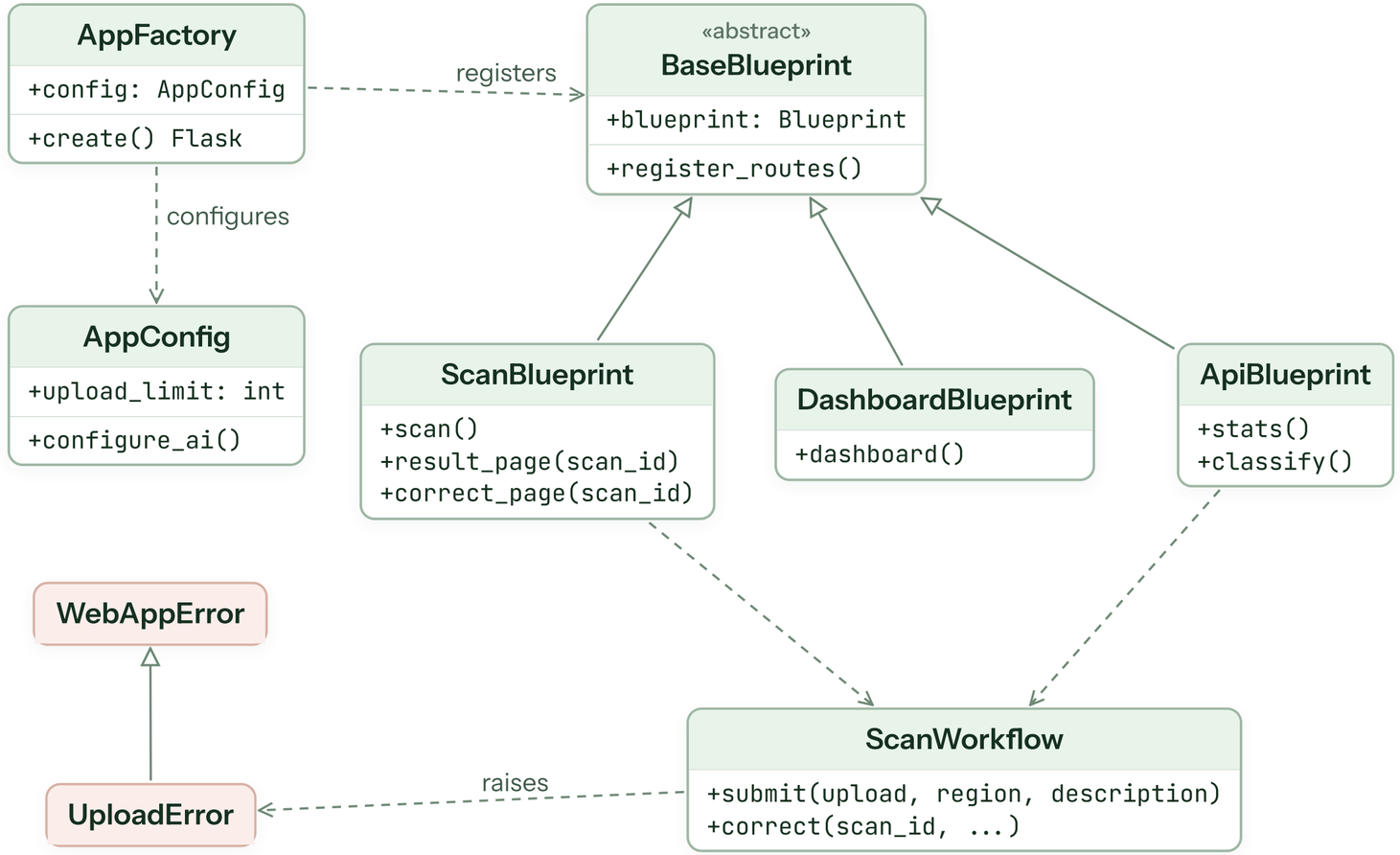
C Dashboard & Analytics · key code

```python
# analytics.py
class StatsCalculator:
    def get_stats(self, region=None, start=None, end=None):
        scans = self._repository.get_scans(
            region=region, start=start, end=end)
        return RecyclingStats.from_scans(scans)

class RecyclingStats:
    @classmethod
    def from_scans(cls, scans):
        stats = cls()
        for scan in scans:
            stats.record_scan(scan)
        return stats

    def increment_category_count(self, category):
        if category not in CATEGORIES:
            raise InvalidAnalyticsInputError(
                f"Unknown category: {category!r}")
        self._category_counts[category] += 1
```





How it works

1. **create_app() builds the app**
   Tests get their own app each time.
2. **Register three blueprints**
   Sorting pages, dashboard, JSON API.
3. **ScanWorkflow checks input**
   City, file type, size, description.
4. **Errors become clean pages**
   Each error class carries its own HTTP status.

```python
# app/__init__.py · AppFactory.create
route_groups: list[BaseBlueprint] = [
    ScanBlueprint(), DashboardBlueprint(), ApiBlueprint(),
]
for group in route_groups:
    app.register_blueprint(group.blueprint)
self._register_error_handlers(app)

# app/routes/scan.py · ScanWorkflow.submit
has_photo = upload is not None and bool(upload.filename)
if not region or (not has_photo and not description):
    raise UploadError("Choose a city and provide a photo and/or item description.")
if region not in self.get_regions():
    raise UploadError("Choose a region from the list ...")
if has_photo:
    extension = upload.filename.rsplit(".", 1)[-1].lower()
    if extension not in current_app.config["ALLOWED_TYPES"]:
        raise UploadError("Choose a supported photo ...")
    image_bytes = ...  # read and size-checked
    result = self._service.classify_upload(image_bytes, region)
```

Shared base class

All route groups inherit from BaseBlueprint.

Input validation

Bad input is rejected before A is called or a scan is saved.

Deployment

With a .env file, docker compose up starts the app and MySQL.

Unit test results

89 / 89

tests pass on main, Python 3.12

80 %

statement and branch coverage

The AI client is mocked and each test uses a temporary database, so tests need no key and no network.

Subsystem	Tests	Coverage of main module
A AI Services	4 · test_ai_services.py	ai_services.py 74 %
B Data & Rules	4 · test_db.py	db.py 80 %
C Analytics	4 · test_analytics.py	analytics.py 70 %
D Web App	77 · test_app.py, test_image_uploads.py	routes/scan.py 93 %

```
$ pytest --cov=app --cov-branch
89 passed in 39.73s · TOTAL 80%
```

Thank you

github.com/Timmyy3000/waste-sorting-assistant

Code, screencast and slides are in the repository. Questions are welcome.